

# Intelligent Web Data Aggregator

## LLM-assisted Web Extraction API

Student: Yan Marchan

Program: Informatics (3rd year)

Supervisor: Evgeniy Tretyak

Jan 2026

# Problem

---

- **Need:** teams need fresh web data (**prices, job posts, news, catalogs, events**).
- **Target users:** analysts, product teams, procurement, recruiters, researchers.
- Modern sites are dynamic (JavaScript rendering) and change layout frequently.
- Classic scrapers rely on brittle patterns ( `CSS/XPath/regex` ) and require constant maintenance.
- **Impact:** stale data and high engineering cost for every new source / redesign.

# Goal & Scope

---

- **Input:** URL + natural language prompt ("extract title and price").
- **Output:** structured JSON stored in DB and accessible via API.
- **Async by design:** create task → poll status/result.
- **Scope boundaries:** public pages; no CAPTCHA solving; one URL per task (no crawler).
- **Automation:** user-owned cron schedules for recurring collection.

# Key Contributions

---

- **Semantic content extraction:** enrich visible text with role markers ( `[LINK]` , `[BUTTON]` , headings, tables, images).
- **Dual extraction paths:** schema (multi-field) and single-field pipelines.
- **LLM-assisted parser generation:** build parsers (regex/CSS/JSONPath) with validation and retries.
- **Parser cache + self-healing:** reuse on cache-hit; auto-invalidate when stale.
- **Production basics:** JWT auth, rate limiting, prompt-injection sanitization, health checks, correlation IDs.

# System Architecture

---

```
Client
|
v
FastAPI API (port 8000) -----> PostgreSQL 15 (tasks, results, cache)
|
v
Redis (Celery broker / rate-limit store)
|
+--> AI Worker (queue: ai_queue) <----- Scheduler (Celery Beat)
|
+--> Headless Worker (queue: fetching_queue, Playwright) ---> Target Websites
```

- **Stack:** FastAPI, Celery, Redis, Postgres, Playwright, LiteLLM (Baseten/DeepSeek, Gemini, optional OpenAI), Docker Compose.

# End-to-End Flow

---

1. **Create task:** `POST /api/v1/process` (JWT required).
2. **Intent extraction:** AI Worker converts prompt → `keywords` / `schema_fields`.
3. **Fetch:** Headless Worker renders page with Playwright and captures semantic text + HTML.
4. **Cache-first:** try validated parsers for the same URL + fields.
5. **If cache miss:** generate a parser (regex/CSS/JSONPath) and validate it before caching.
6. **Persist result:** save JSON + metadata; client polls `/api/v1/status` and `/api/v1/result`.

## Appendix: End-to-End Flow (Notes)

---

- **Cost control:** LLM sees focused snippets (not full HTML); parsers run locally on full content.
- **Traceability:** intent + sources are stored; correlation IDs help follow logs end-to-end.
- **Cache behavior:** cache hit skips parser generation; stale parsers are invalidated and regenerated.

# API Interface

---

- **Docs:** Swagger UI at `/docs`
- **Auth:** `POST /auth/register`, `POST /auth/token` → Bearer token
- **Tasks:** `POST /api/v1/process`, `GET /api/v1/status/{task_id}`, `GET /api/v1/result/{task_id}`
- **Scheduling:** `POST/GET/DELETE /api/v1/jobs`
- **Rate limits:** process 10/min, register 5/min, login 10/min (per IP)



## Appendix: API Example (Create Task)

---

```
POST /api/v1/process
{
  "url": "https://example.com/page",
  "prompt": "Extract product title and price"
}
→ 202 { "task_id": "...", "status": "PENDING" }
```

# Headless Worker (Playwright)

---

- **Purpose:** render JavaScript-heavy pages and capture extraction-friendly signals.
- **Captured artifacts:**
  - `inner_text`: visible body text
  - `semantic_text`: enriched text with markers ( `[LINK]` , `[BUTTON]` , headings, tables, image alt/URLs)
  - `html`: full DOM snapshot (stored with defensive cap)
  - `network`: selected XHR/fetch/document/script events (preview)
- **Stealth basics:** randomized UA/viewport, init script, cookie banner dismissal attempts.

# Intent Extraction & Safety

---

- **Prompt-injection defense:** sanitize user input before queueing work (defense in depth).
- **Intent extraction:** LLM converts prompt into structured intent:
  - `keywords` (single-field extraction)
  - `schema_fields` (multi-field structured extraction)
  - `target` (what the user wants overall)
- **Normalization:** field names are normalized for consistency ( `"Price"` → `price` ).
- **Traceability:** intent is stored with the task to explain “why this result”.

# Extraction Pipeline

---

- **Cache-first:** reuse validated parsers for the same URL + fields.
- **Two paths:**
  - **Schema extraction:** LLM returns structured items with field associations.
  - **Single-field extraction:** find examples → locate snippets → generate parser → validate.
- **Parser types:** regex (common), CSS selector, JSONPath (when applicable).
- **Quality gate:** generate → execute → validate → retry; only validated parsers are cached.
- **Token optimization:** regex generation uses focused snippets (e.g.,  $\leq 32k$  chars), not full HTML.

# Criterion: Back-end

---

**WHY:** Long-running extraction needs an async API with a reliable task lifecycle and clear contracts.

**WHAT:**

- FastAPI endpoints for auth, task creation, status polling, results, and cron jobs.
- Async orchestration: API → Celery queues → workers → persisted result JSON.
- Validation + rate limits + correlation IDs for safe, traceable execution.
- JWT-protected access to user-owned tasks and schedules.

**TECH:** FastAPI, Uvicorn, Pydantic, Celery, Redis, SlowAPI, JWT.

```
POST /api/v1/process → task_id
GET /api/v1/status/{task_id}
GET /api/v1/result/{task_id}
POST /api/v1/jobs (cron)
```

# Criterion: AI assistant / Chatbot

---

**WHY:** Web pages change frequently; intent-driven extraction reduces brittle selector/regex maintenance.

## **WHAT:**

- Prompt → intent ( `schema_fields` / `keywords` ) + normalization.
- LLM-assisted extraction using semantic text + focused snippets (cost control).
- Parser generation (regex/CSS/JSONPath) with validation + retries.
- Cache + self-healing: reuse on hit, invalidate + regenerate on failure.

**TECH:** LiteLLM (Baseten/DeepSeek, Gemini fallback), prompt templates, local parser execution/validation.

```
prompt → intent(fields)
semantic_text/html → LLM extract examples/items
→ generate parser → validate → cache(url + fields)
→ run parser locally on full content
```

# Criterion: Database

---

**WHY:** Need ACID storage for users, tasks, results, schedules, and cached parsers (plus captured page artifacts).

## **WHAT:**

- Store users, tasks, results, schedules, and extraction metadata.
- Persist captured artifacts (semantic text, HTML, network previews) with defensive truncation.
- Parser cache indexed by **domain + full URL + fields** for reliable cache hits.

**TECH:** PostgreSQL 15, SQLAlchemy, Alembic migrations.

```
ScrapingTasks → TaskSourceData (semantic_text/html/network)
Users → ScheduledJobs (cron)
ParserCache key: (domain, full_url, fields, source_type)
```

# Appendix: Database Entities

---

| Entity         | Purpose                                  |
|----------------|------------------------------------------|
| Users          | Authentication + ownership               |
| ScrapingTasks  | Status, prompt, extracted JSON, metadata |
| TaskSourceData | Semantic text / HTML / network previews  |
| ScheduledJobs  | Cron schedules per user                  |
| ParserCache    | Reusable validated parsers + samples     |



# Criterion: Microservices

---

**WHY:** Separate fetching/rendering and LLM work from the API to scale independently and isolate failures.

## WHAT:

- API service handles auth + task creation and exposes OpenAPI/Swagger.
- Headless Worker renders pages (Playwright) and captures semantic text + HTML.
- AI Worker runs intent/extraction/parsing and updates the DB.
- Scheduler (Celery Beat) enqueues due cron jobs.

**TECH:** Celery (queues: `fetching_queue`, `ai_queue`), Redis, Postgres, Playwright.

```
API → Redis/Celery → Headless Worker → AI Worker → Postgres
      Scheduler (Beat) enqueues recurring jobs
```

# Criterion: Automated tests $\geq$ 70% coverage

---

**WHY:** Prevent regressions in API contracts and extraction workflows across multiple services.

**WHAT:**

- Pytest suite covering unit + integration + API contract/workflow tests.
- Coverage tracked with `pytest-cov`; baseline above the requirement.
- Tests validate critical metadata like `used_cached_parser`.

**TECH:** Pytest, pytest-cov.

Coverage summary (pytest-cov):

TOTAL 4168 statements, 926 missed → 78% covered

# Criterion: Containerization

---

**WHY:** Reproducible environment for multi-service runtime (DB/Redis/workers) and a predictable demo setup.

**WHAT:**

- Each service runs in its own container (API, headless worker, AI worker, scheduler).
- Compose orchestrates dependencies, environment variables, and health checks.
- Isolation for Playwright runtime and consistent execution across machines.

**TECH:** Docker, Docker Compose.

```
docker compose up -d  
docker compose ps  
docker compose logs -f api
```

# Appendix: Docker Compose Services

---

```
services:
  postgres: postgres:15-alpine (healthcheck)
  redis:    redis:7-alpine      (healthcheck)
  api:      FastAPI (8000)
  headless_worker: Celery -Q fetching_queue (Playwright)
  ai_worker:      Celery -Q ai_queue (LLM + parsing)
  scheduler:      Celery Beat
```

# Criterion: API documentation

---

**WHY:** A stable API contract and interactive UI speeds up integration and demo walkthroughs.

**WHAT:**

- OpenAPI schema generated automatically from FastAPI routes and models.
- Swagger UI supports auth + end-to-end calls (process → status → result).
- Examples embedded in docs for predictable client integration.

**TECH:** FastAPI OpenAPI + Swagger UI ( `/docs` ).

```
GET /docs          # Swagger UI
GET /openapi.json  # OpenAPI schema
Authorize: Bearer <token>
Try it out: /process → /status → /result
```

# Appendix: Swagger UI Screenshot

## Intelligent Web Data Aggregator 1.0.0 OAS 3.1

/openapi.json

An intelligent service that accepts a URL and a natural language prompt and automatically extracts the requested data using web scraping and Large Language Models (LLMs).

## Features

- **Natural Language Processing**: Use plain English to describe what data you want
- **Intelligent Extraction**: LLM-powered data identification and extraction
- **Regex Generation**: Automatic regular expression creation and validation
- **Caching**: Smart caching of successful parsers for future use
- **Asynchronous Processing**: Non-blocking task execution with status tracking

Contact Yan Marchan  
MIT

Authorize 

**Root** Basic information and navigation for the API. ^

GET / Root v

**Health** Liveness and readiness probes, including DB connectivity checks. ^

GET /health Health Check v

GET /api/v1/health Api Health Check v

**Auth** Authentication endpoints (login, register). ^

POST /auth/register Register v

POST /auth/token Login For Access Token v

# Demo Screenshots (1/3) — Authentication

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/auth/register' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "marchie",
    "password": "ThisIsMyStrongPassword",
    "email": "20220553@student.ehu.lt"
  }'
```

Request URL

http://127.0.0.1:8000/auth/register

Server response

| Code | Details |
|------|---------|
|------|---------|

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200 | <div>Response body</div> <div><pre>{   "id": 12,   "username": "marchie",   "email": "20220553@student.ehu.lt",   "is_active": true }</pre></div> <div>Response headers</div> <div><pre>access-control-allow-credentials: true access-control-allow-origin: * content-length: 81 content-type: application/json date: Thu, 08 Jan 2026 16:14:44 GMT server: uvicorn x-correlation-id: b0a80e12-dd9b-4e36-bf2a-eda959f8b83e</pre></div> |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Available authorizations

x

Scopes are used to grant an application different levels of access to data on behalf of the end user. Each API may declare one or more scopes.

API requires the following scopes. Select which ones you want to grant to Swagger UI.

### OAuth2PasswordBearer (OAuth2, password)

Authorized

Token URL: /auth/token

Flow: password

username: marchie

password: \*\*\*\*\*

Client credentials location: basic

client\_secret: \*\*\*\*\*

Logout

Close

- **Flow:** register → login → get JWT token
- **Show:** Swagger “Authorize” with Bearer token
- **Optional:** invalid login error state

## Demo Screenshots (2/3) — Create Task & Poll Status

[illegible]

Request URL

**http://127.0.0.1:8000/api/v1/process**

### Server response

| Code | Details |
|------|---------|
|------|---------|

202

Response body

```
{
  "task_id": "d9dbc2eb-c75d-42d6-a284-897fa743ef99",
  "status": "PENDING",
  "message": "Task created successfully"
}
```

### Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: *
content-length: 107
content-type: application/json
date: Thu, 08 Jan 2026 16:32:21 GMT
server: uvicorn
x-correlation-id: af1e1b77-c5be-4de1-92fa-a1b9336862a8
```

Curl

```
curl -X 'GET' \  
  'http://127.0.0.1:8000/api/v1/status/d9dbc2eb-c75d-42d6-a284-897fa743ef99' \  
  -H 'accept: application/json' \  
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJtYXJa
```

Request URL

<http://127.0.0.1:8000/api/v1/status/d9dbc2eb-c75d-42d6-a284-897fa743ef99>

### Server response

| Code | Details |
|------|---------|
|------|---------|

200

Response body

```
{
  "task_id": "d9dbdc2eb-c75d-42d6-a284-897fa743ef99",
  "status": "IN_PROGRESS",
  "progress": 50,
  "message": "Task is in progress",
  "created_at": "2026-01-08T16:32:22.195892Z",
  "updated_at": "2026-01-08T16:32:34.049377Z"
}
```

### Response headers

```
content-length: 205
content-type: application/json
date: Thu, 08 Jan 2026 16:32:55 GMT
server: uvicorn
x-correlation-id: 46645385-7d56-4605-a8bf-a1e4a7a8980a
```

- **Input:** URL + natural language prompt
- **Response:** 202 Accepted + `task_id`
- **Status gathering:** poll `/api/v1/status/{task_id}` until `SUCCESS`



## Demo Screenshots (3/3) — Fetch Result

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/api/v1/result/d9dbc2eb-c75d-42d6-a284-897fa743ef99' \
-H 'accept: application/json' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpvcCJ9.ejYzdWl0iojTYXJlaGllIiwizXhwIjoNzY3ODkxMDMwQ0f0.1iaJ_Q64P0JmHLAgU_F
```

Request URL

<http://127.0.0.1:8000/api/v1/result/d9dbc2eb-c75d-42d6-a284-897fa743ef99>

### Server response

| Code | Details |
|------|---------|
|------|---------|

200

Response body

```
{
  "task_id": "d9dbc2eb-c75d-42d6-a284-897fa743ef99",
  "status": "SUCCESS",
  "url": "https://puko.lt/ru/xalati-zhenskie",
  "prompt": "items name, prices, and links",
  "data": [
    {
      "text": "name: Халат Бамбоо весна | price: 75.00€ | link: https://puko.lt/ru/xalaty-dlinyje-kapishon-vesna",
      "fields": {
        "name": "Халат Бамбоо весна",
        "price": "75.00€",
        "link": "https://puko.lt/ru/xalaty-dlinyje-kapishon-vesna"
      },
      "source": "schema_extraction",
      "confidence": 0.9
    },
    {
      "text": "name: Халат Бамбоо Копи | price: 54.00€ | link: https://puko.lt/ru/xalaty- kapishon-bambuk-copi",
      "fields": {
        "name": "Халат Бамбоо Копи",
        "price": "54.00€",
        "link": "https://puko.lt/ru/xalaty- kapishon-bambuk-copi"
      },
      "source": "schema_extraction",
      "confidence": 0.9
    },
  ]
}
```

- **When ready:** `GET /api/v1/result/{task_id}`
- **Data:** structured items with fields (e.g., name/price/link)
- **Metadata:** `total_matches` + `used_cached_parser`

# Challenges & Solutions

---

| Problem                                | Solution                                                           | What I learned                                                      |
|----------------------------------------|--------------------------------------------------------------------|---------------------------------------------------------------------|
| Dynamic JS pages + layout drift        | Playwright render + semantic capture + validated parser cache      | Make extraction robust by separating “understanding” from “parsing” |
| Prompt injection / unsafe instructions | Input sanitization + defense-in-depth prompts + strict pipeline    | Security must be built into the workflow, not added later           |
| Cached parser becomes stale            | Self-healing: invalidate on failure and regenerate with validation | Monitoring + feedback loops keep automation reliable                |

# Results

---

| Capability                                   | Status  |
|----------------------------------------------|---------|
| Async task API (process → status → result)   | ✓       |
| Playwright rendering + semantic text capture | ✓       |
| LLM intent extraction + schema extraction    | ✓       |
| Validated parser cache + self-healing        | ✓       |
| User-owned scheduling (cron jobs)            | ✓       |
| Testing coverage ≥ 70%                       | ✓ (78%) |

- **Deployment:** N/A (local demo via Docker Compose)
- **Backlog:** UI dashboard, multi-URL crawling, stronger anti-bot strategies

# Thank You

## Q & A

**Production URL:** N/A (not deployed; demo runs locally)

**Repository:** N/A (private/local) — archive: `Marchan_Yan_Diploma_upd.zip`

**API Docs (Swagger):** `http://localhost:8000/docs`

**Project docs:** `docs_old/ARCHITECTURE.md`, `docs_old/USER_GUIDE.md`

**Design:** N/A